

PawGuard — Complete System Architecture

PAWGUARD-SYSARCH-01

End-to-End Backend Design: Client Apps, API, Routing, Services, Cache, Queue & Database
PawGuard Rescue & Adoption Operations

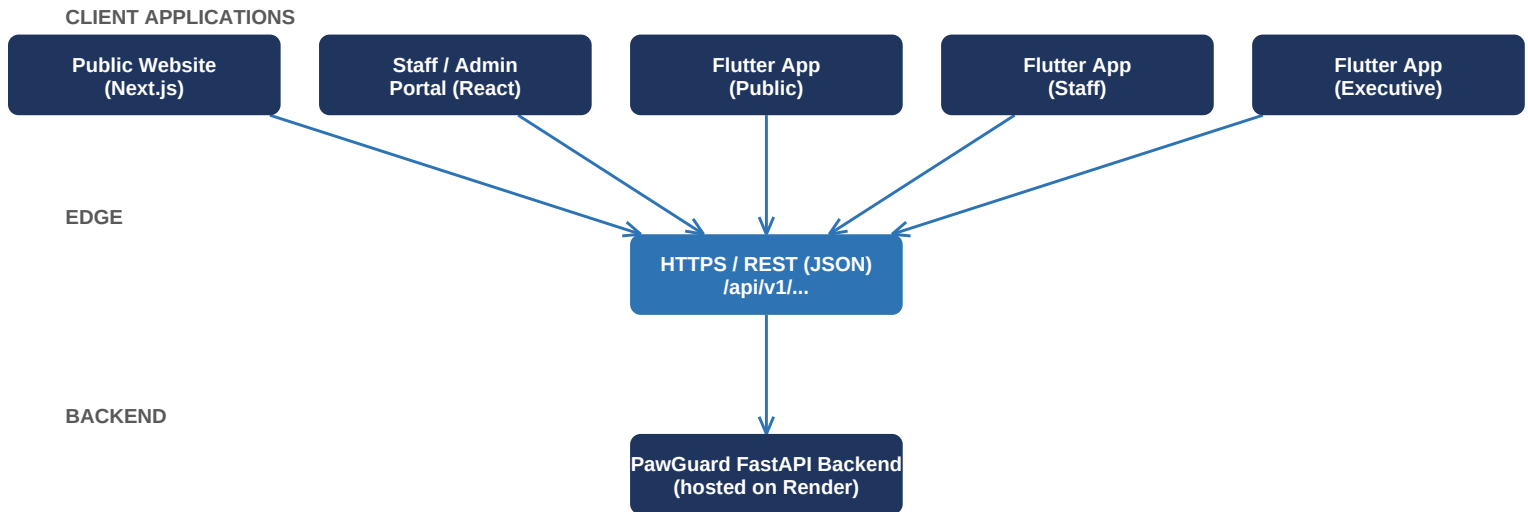
August 2026

1. What This Document Covers

A single, complete picture of how the PawGuard backend actually works: every client that calls it, every layer a request passes through, exactly where Redis and the database sit in that path, how all 23 domain modules relate to one another, and how background jobs run outside the request/response cycle. Every fact below is taken directly from the pawguard-backend source code (main.py, core/, modules/*) — nothing here is guessed.

2. System Context — Who Calls This Backend

One FastAPI backend is the single source of truth for every PawGuard-facing application:



Every client — the website, the staff portal, and all three Flutter apps — talks to the exact same versioned REST API. There is no separate backend per client; the authenticated user's **role** (owner, vet, volunteer, staff, admin) is what changes what they can see and do, not which API they call.

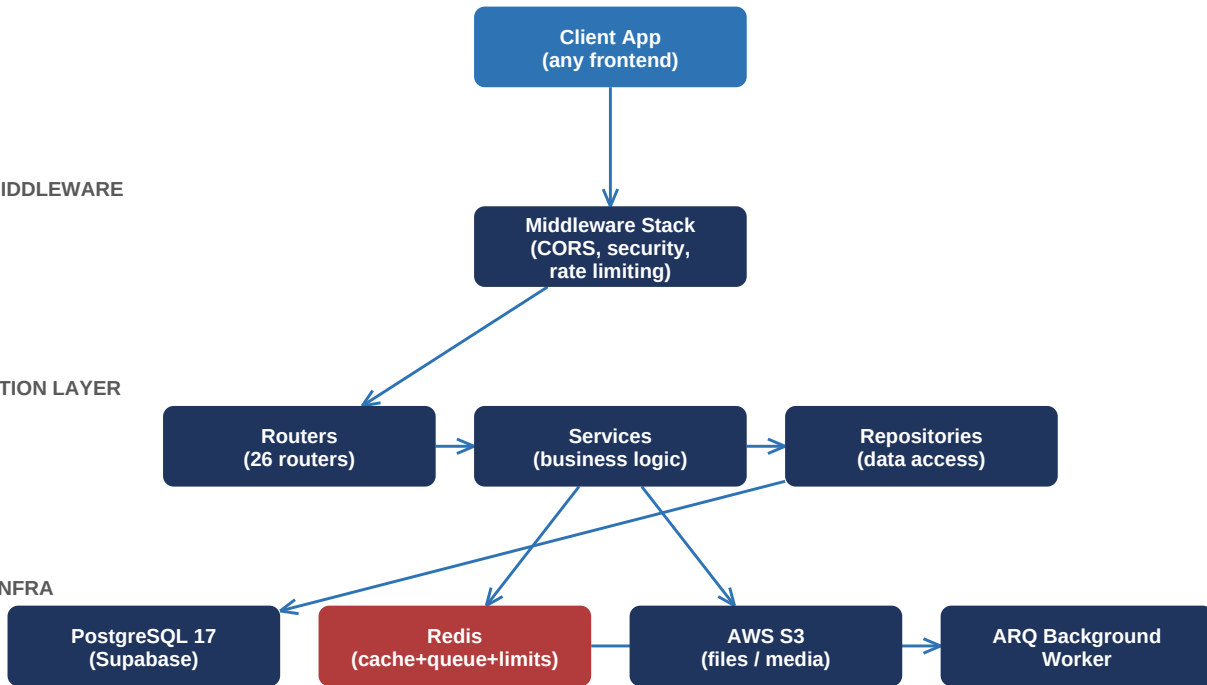
3. High-Level Architecture — All the Moving Parts

CLIENT LAYER

EDGE / MIDDLEWARE

APPLICATION LAYER

DATA & INFRA



- **Routers** authenticate, authorize, validate input, and shape responses — they never contain business rules.
- **Services** own 100% of business logic and are the only layer allowed to make business decisions.
- **Repositories** perform data access only — async SQLAlchemy 2.x queries against PostgreSQL.
- **Redis** is used for three distinct jobs (Section 5): permission caching, rate limiting, and the ARQ background-job queue — it sits beside the service layer, never between the client and the API.

4. Request / Response Lifecycle — Step by Step

The exact path a single API call takes, end to end, per main.py's middleware registration order:



If step 7 or 8 fails, the request is rejected (401 / 403) before it ever reaches business logic — individual services never have to re-check identity or permissions themselves.

5. Where Redis Actually Sits

Redis is **not** a hop between the frontend and the API — it's an internal service the backend process talks to, for three unrelated jobs:

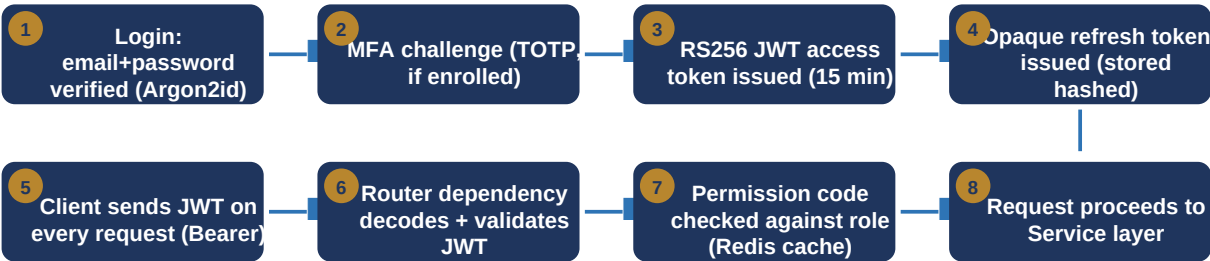
Use	What It Does	Touches Which Layer
RBAC/PBAC permission cache	Caches a role's resolved permission codes for 5 minutes so most requests don't re-query Postgres for access checks	Auth dependency, before the endpoint body runs
Rate limiting	Redis INCR on a per-user/IP bucket key with a TTL window — e.g. login 10/60s, QR-tag scan 20/60s, lost-pet broadcast 3/hour	A FastAPI dependency attached to specific sensitive endpoints
ARQ job queue	Background jobs (email delivery, reminders, lost-pet broadcast fan-out, report generation) are pushed onto a Redis-backed queue and picked up by the ARQ worker	Called from inside Services; consumed by the separate worker process

Accurate mental model: **Client** → **API (Routers/Services)** ↔ **Redis** (cache/queue, purely internal) and separately **API** → **PostgreSQL** (source of truth). Redis is a fast internal helper the API leans on — the frontend never talks to Redis directly.

Reference: `main.py` (middleware order), `core/security.py` (JWT), `core/rate_limiter.py`, `modules/auth/rbac.py` (Redis-cached RBAC).

6. Authentication & Authorization — Detail

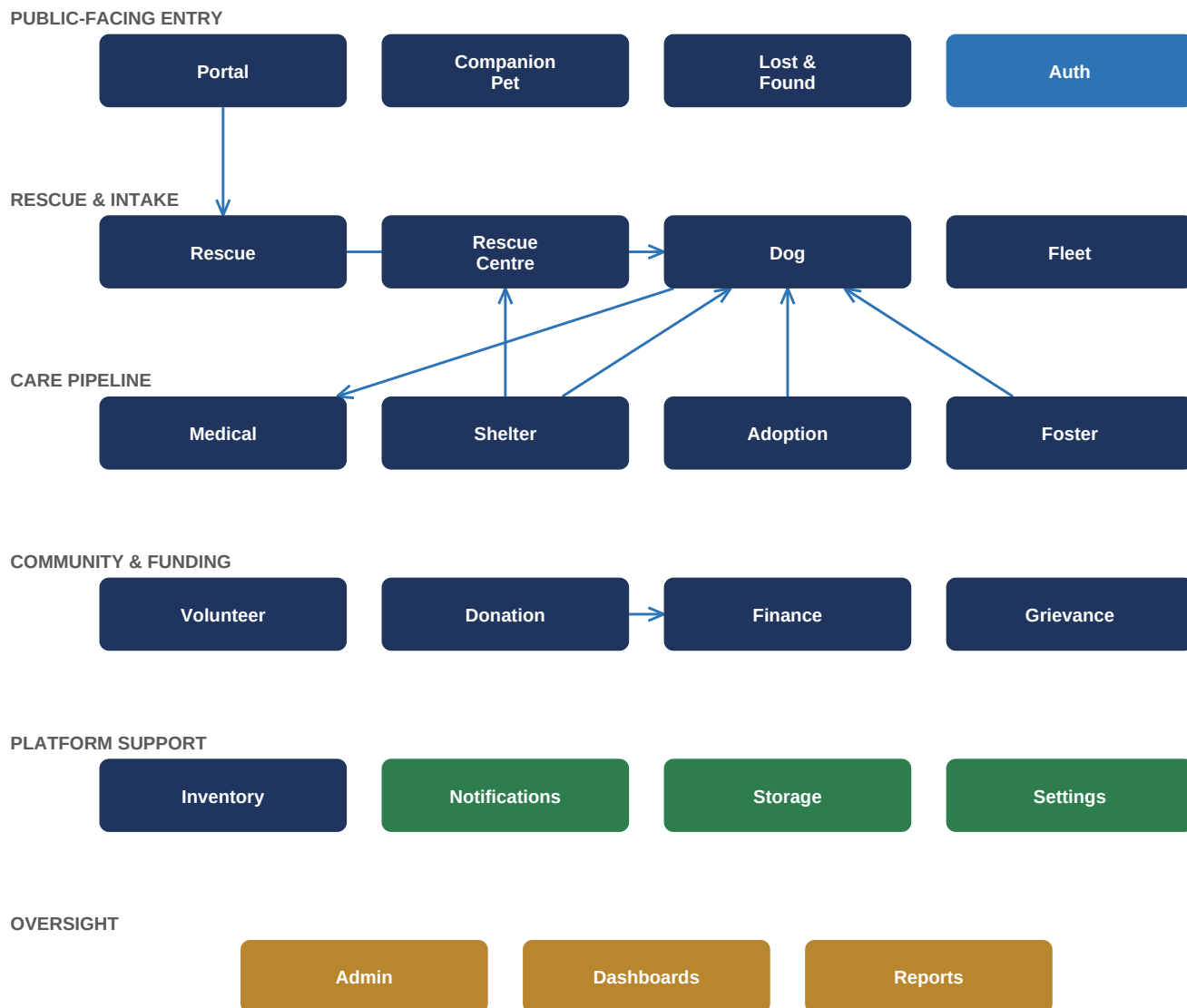
Every one of the 26 routers depends on the same two primitives, so this flow effectively “connects” every module to the auth module:



- Access tokens are short-lived (15 min) and signed with RS256 (private key signs, public key verifies) — no shared secret needs to live in every service.
- Refresh tokens are opaque, hashed at rest (SHA-256), and rotated on use; reuse of an already-rotated refresh token is treated as a compromise signal and logged (REFRESH_REUSE_DETECTED).
- Every login, logout, refresh, and failed attempt is written to the central AuthAuditEventType log, which the Admin module exposes to staff.

7. All 23 Domain Modules & How They Connect

23 domain modules sit behind 26 routers. Grouped by function, with the real cross-module integrations the code implements (arrows) — modules not connected by an arrow below still share the cross-cutting services in Section 8 (auth, notifications, storage, settings).



Longer-reach integrations that don't fit cleanly on the diagram above (they cross multiple lanes):

From → To	What Happens
Portal → Adoption / Volunteer / Donation	Public submissions from the website are routed straight into each owning module
Companion Pet → Storage	Medical-file uploads use the shared presigned-S3 upload flow
Companion Pet → Notifications	Vaccination/medication reminders are delivered via the shared notification service
Lost & Found → Notifications	Broadcast alerts are delivered via the shared notification service
Inventory → Foster	Supply items (food, crate, medication, bedding) are issued against a foster placement

- **Rescue → Dog:** an admitted rescue case becomes a formal Dog record (status = ADMITTED → tracked from there).
- **Adoption / Foster → Dog:** approval flips the dog's status to ADOPTED / FOSTERED.
- **Shelter → Rescue Centre:** the rescue-centre directory is built directly on shelter facility data.
- **Donation → Finance:** a successful donation auto-posts to the finance ledger.
- **Companion Pet → Storage / Notifications:** medical-file uploads use the storage module; reminders use notifications.
- **Lost & Found → Notifications:** broadcast alerts are delivered through the shared notification service.
- **Portal** is the public front door for rescue reports, adoption applications, volunteer sign-ups, and donations.

8. Cross-Cutting Services (Touch Nearly Every Module)

Service	Who Calls It	Purpose
Auth (RBAC/PBAC)	All 26 routers	Identity + permission gate on every request
Notifications	companion_pet, lost_found, adoption, donation, grievance, and more	In-app + email delivery, sync and async
Storage (S3)	companion_pet, dog, medical, rescue, reports	Presigned-URL uploads for every module that accepts files
Settings	Any module reading a feature flag or global config	Central configuration, read at runtime
Admin / Dashboards / Reports	Reads across nearly all modules	Aggregation, audit, and export — read-mostly, no owned business tables

Reference: `modules/*/service.py` cross-module calls, `src/pawguard/api/v1/router.py` (26 `include_router` calls).

9. Background Jobs — What Happens Outside the Request/Response Cycle

The ARQ worker runs as a second process (or a concurrent asyncio task in the same container on Render's free tier) consuming jobs from Redis — on a schedule (cron) and on demand (queued by a service call):

Schedule	Job	What It Does
Daily 00:00 & 12:00	<code>_check_inventory_low_stock</code>	Flags supplies under reorder threshold
Daily 09:00	<code>_check_inventory_expiry</code>	Flags supplies nearing expiry
Daily 09:30	<code>_check_vaccination_renewals</code>	Finds shelter dogs due for a vaccine renewal
Daily 09:45	<code>_send_companion_pet_reminders</code>	Delivers due vaccination/medication reminders to pet owners
Daily 10:00	<code>_post_adoption_followups</code>	Creates scheduled post-adoption follow-up check-ins
Daily 08:00	<code>_process_sponsorship_charges</code>	Processes recurring sponsorship/donation charges
On demand (queued)	<code>broadcast_lost_pet_alert</code>	Fans out a lost-pet alert (rate-limited 3/hour at the API layer)
On demand (queued)	<code>send_email / send_notification</code>	Transactional email (Brevo HTTP API) + in-app notifications
On demand (queued)	<code>generate_report</code>	Renders CSV/XLSX/PDF exports into <code>generated_reports/</code>

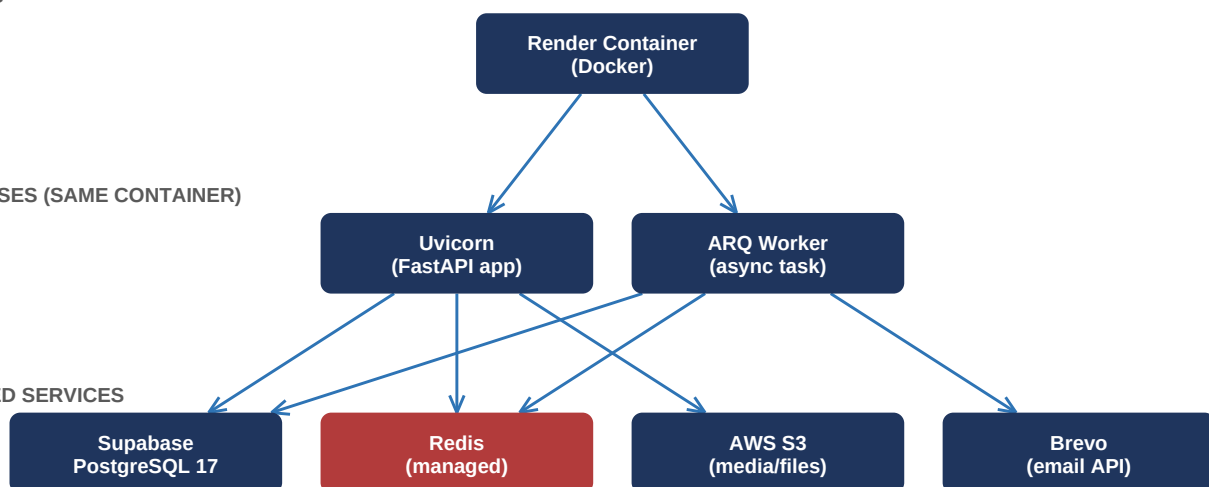
If Redis is temporarily unreachable, the API still serves requests — background jobs degrade to a no-op rather than crashing the service (self-healing on worker restart for stale ARQ locks).

10. Deployment Architecture

HOSTING

PROCESSES (SAME CONTAINER)

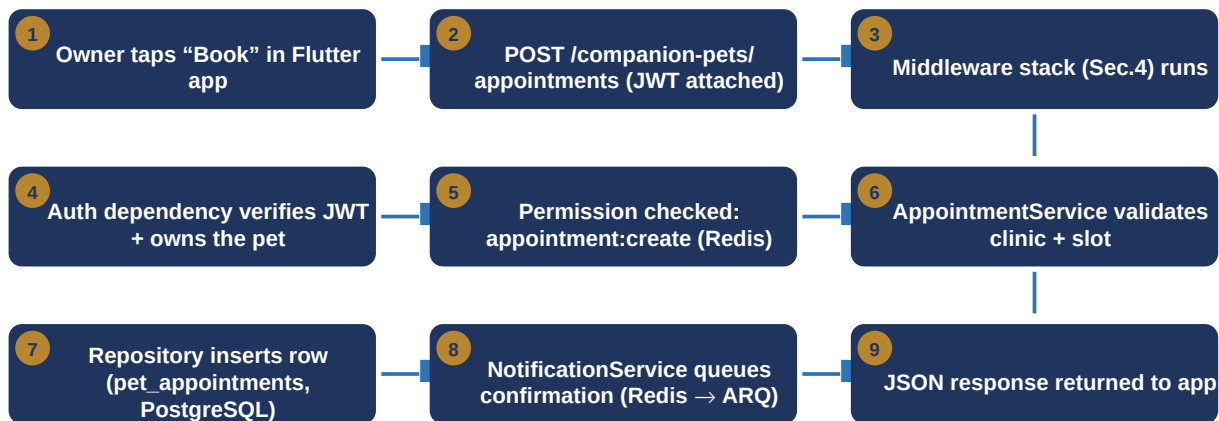
MANAGED SERVICES



- On Render's free tier, the API and the ARQ worker run as concurrent asyncio tasks inside one process/container — no paid separate Background Worker service is required.

- Health endpoints (/health, /live, /ready) let Render's load balancer and any external uptime monitor verify both the app and its DB/Redis dependencies are healthy.

11. Worked Example — “Owner Books a Vet Appointment” End to End



This single trace touches five different modules (companion_pet, auth, notifications) and two infrastructure pieces (Redis, PostgreSQL) — which is exactly what “how it all connects” means in practice: no module works in isolation, but each layer only knows about the layer directly below it.

Reference: pawguard-backend engineering documentation — complete system architecture, compiled from main.py, core/, and modules/ across the full backend.*